

Практическая работа 4 по Информационным технологиям

Обращение к памяти, переходы, ветвления, циклы

Сначала поясним некоторые особенности работы с памятью в ассемблере. Чтобы ассемблер отличал данные от исполняемого кода, существуют специальные директивы размещения данных в памяти. Разные функциональные области памяти называются сегментами или секциями.

Для этого код программы помечается директивой `.text`, а данные — `.data`. При трансляции в Ripes код размещается с адреса `0x00000000`, а данные — с адреса `0x10000000`, если не сказано иное.

В секции `.data` помещают директивы (указания ассемблеру) по размещению данных в памяти:

`.word` число — одно или несколько 4-байтовых чисел

`.dword` число — одно или несколько 8-байтовых чисел

`.half` число — одно или несколько 2-байтовых чисел

`.byte` число — одно или несколько однобайтовых чисел

`.asciz` "строка" — последовательность символов в кодировке ASCII, после последнего символа обязательно записывается нулевой байт (конец строки, договорённость, например, для языка Си).

Пример 1 размещения данных различного размера:

```
.data
    .word    0xdeadbeef
    .dword   0xacebad0feeded
    .half    0x1234, 0x5678
    .byte    12, 13, 14, 15
    .half    0x3344
    .byte    0x66, 0x77

.text
_start:
    addi s3, zero, 10
```

Можно чередовать объявления секций, то есть директивы `.data` перемежать с директивами `.text`, ассемблер всё рассортирует в памяти как надо.

Если мы загрузим наш пример размещения данных в Ripes, то в разделе Memory увидим картину, изображённую на табл.1. Адреса в этой таблице возрастают снизу вверх. На первом адресе `0x10000000` для сегмента `.data` (самая последняя строка) расположено слово `0xdeadbeef`. Размещение по байтам начинается с младшего байта `0 – 0xEF`, затем следует байт `1` со значением `0xBE`, потом байт `2 – 0xAD`, и, наконец, самый старший байт `3` со значением `0xDE`. Этот порядок размещения данных по байтам определен в архитектуре RISC-V как Little-Endian. Обратите внимание на первую строку с данными по адресу `0x10000014`: `0x44, 0x33, 0x66, 0x77`. Так как директива `.half` загружает только половину

слова, то оставшиеся два байта занимают данные уже из следующей директивы .byte 0x66 и 0x77. То есть данные размещаются в памяти последовательно и побайтно.

Таблица 1. Память сегмента .data

Memory viewer					
Address	Word	Byte 0	Byte 1	Byte 2	Byte 3
0x1000001c	X	X	X	X	X
0x10000018	X	X	X	X	X
0x10000014	0x77663344	0x44	0x33	0x66	0x77
0x10000010	0x0f0e0d0c	0x0c	0x0d	0x0e	0x0f
0x1000000c	0x56781234	0x34	0x12	0x78	0x56
0x10000008	0x000aceba	0xba	0xce	0x0a	0x00
0x10000004	0xd0feeded	0xed	0xed	0xfe	0xd0
0x10000000	0xdeadbeef	0xef	0xbe	0xad	0xde

Разберём, как работают псевдоинструкции li (load immediate) и la (load address):

```
.data
var:    .word    0xbadface

.text
    li     t5 0x10000008
    la     t6 var
```

Несмотря на то, что метка var соответствует адресу 0x10000000, и значения регистров t5 и t6 будут равны (проверьте это), действия эти псевдоинструкции задают разные. Посмотрим на получившийся код:

```
0:      10000f37      lui x30 0x10000
4:      008f0f13      addi x30 x30 8
8:      10000f97      auipc x31 0x10000
c:      ff8f8f93      addi x31 x31 -8
```

Псевдоинструкция li t5 0x100100ff превратилась в загрузку старших 20 битов числа 0x10000 с помощью инструкции типа U (lui) и добавление младших 12 (число 8).

Псевдоинструкция la t6 var сначала воспользовалась другой инструкцией типа U — auipc.

Инструкция «auipc регистр, смещение» работает так:

- как полагается инструкции U, заполняет 20 старших битов регистра смещением. Младшие 12 битов зануляет. В нашем случае получается 0x10000.

- добавляет в регистр адрес текущей исполняемой инструкции. В нашем случае — 0x00000008. Получается 0x10000008.

Такой подход называется позиционно-независимым кодированием. Смысл его в том, что одну и ту же программу можно загрузить в произвольный адрес. При этом в регистр t5 попадёт число 0x10000008 — потому что именно число мы и хотели туда положить, а в t6 — адрес, отстоящий от данной инструкции на 8 байтов, каким бы ни был её адрес.

Регистр pc не является регистром общего назначения, но для ориентации в программах на RISC-V его всегда надо иметь в виду. Получить значение pc можно, например, с помощью `auipc t0 0`; больше ничего напрямую с ним делать нельзя.

Базовые инструкции вида «сравнить и перейти» — это `beq/bne` и `blt/bge` с их беззнаковыми аналогами `btlu/bgeu`. Все остальные инструкции вида `b*`, включая безусловный переход `b`, — псевдо, потому что получаются простой перестановкой регистров-операндов или подстановкой регистра `zero (x0)` в нужное место. Непосредственный 12-битный операнд используется для хранения смещения относительно текущего адреса. Вычислением этого смещения из метки занимается ассемблер, оно бывает положительное (вперёд, как в примере выше) и отрицательное (назад, как в примере ниже). 12 битов хватает на то, чтобы сделать переход на 4 килобайта кода (4, а не 2, потому что адрес перехода всегда кратен 2 и самый младший бит просто не хранится).

Пример 2 простого цикла с проверкой условия:

```
        li s2 10          # Граница счётчика
        li s1 1            # Счётчик
loop:    addi s1, s1, 1     # Увеличение
        blt s1, s2, loop  # Сравнение счётчика и границы, переход
```

Загрузите этот пример 2 в Ripes и пройдите его по шагам. В окне дизассемблера мы увидим такой код:

```
0:      01100913    addi x18 x0 17
4:      00100493    addi x9 x0 1

00000008 <loop>:
8:      00148493    addi x9 x9 1
c:      ff24cee3    blt x9 x18 -4 <loop>
```

Разберём этот пример 2 подробнее. В начале цикла мы увеличиваем на 1 счётчик цикла. Следующей командой `Branch if Less Than Goto Loop` - если счётчик меньше границы, мы переходим назад на метку `Loop`. А цикл получается типа `do-until`. То есть сначала увеличиваем счётчик на 1, затем проверяем условие и переходим на начало цикла.

При подготовке этого цикла мы загрузили в `s2` значение максимальной границы счётчика, а в `s1` — начальное значение счётчика.

Обратите внимание на переход на -4 байтов назад — это как раз адрес метки `loop`.

Если надо перейти дальше, чем это допустимо в `b*`, используйте отдельно сравнение, безусловный переход псевдоинструкцией `j`: она имеет тип `U/J`, и поле непосредственной константы занимает 20 битов. Но для этого надо написать не менее 4 кБайт кода в одном из условий. Совет: старайтесь так не делать.

Цикл с постусловием — штука ненадёжная, так что идеологически верно было бы переписать пример выше в соответствии с «канонической схемой цикла»:

1. Инициализация
2. Проверка условия
3. Тело
4. Изменение

При этом придётся сделать ещё один (безусловный) переход. Пример 3:

```

                li      s2, 10
                li      s1, 1          # Инициализация
loop:          bge     s1, s2, final  # Проверка условия
                li      a7, 11        # Тело цикла
                li      a0, 10
                addi     s1, s1, 1     # Изменение счётчика цикла
                j        loop         # Дополнительный переход
final:         nop

```

Безусловный переход может называться j, а может jal — это одна и та же псевдоинструкция.

Теперь познакомимся с базовой (косвенной) адресацией и массивами.

Ещё один способ обращаться к памяти — это записать полный абсолютный адрес в регистр, и воспользоваться инструкцией, которая работает с памятью, находящейся по этому адресу. Вот как раскрываются псевдоинструкции lw регистр метка и sw регистр метка вспомогательный регистр:

```

.data
        .word    0x1223344
var:    .word    0xdeadbeef
addr:   .word    var
.text
        lw       t1 var
        lw       t2 addr
        lw       t3 0(t2)
        lw       t4 4(t2)
        lw       t5 -4(t2)
        sw       t5 var t0

```

В коде:

Address	Code	Basic	Line	Source
0x00400000	0x0fc10317	auipc x6,0x0000fc10	6	lw t1 var
0x00400004	0x00432303	lw x6,4(x6)		
0x00400008	0x0fc10397	auipc x7,0x0000fc10	7	lw t2 addr
0x0040000c	0x0003a383	lw x7,0(x7)		
0x00400010	0x0003ae03	lw x28,0(x7)	8	lw t3 (t2)
0x00400014	0x0043ae83	lw x29,4(x7)	9	lw t4 4(t2)
0x00400018	0xffc3af03	lw x30,0xffffffffc(x7)	10	lw t5 -4(t2)
0x0040001c	0x0fc10297	auipc x5,0x0000fc10	11	sw t5 var t0
0x00400020	0xffe2a423	sw x30,0xffffffffe8(x5)		

Сначала идёт уже знакомая нам инструкция `auipc`, которая формирует в регистре `t1` (он же `x6`) адрес, по которому лежит интересующее нас значение.

Затем `lw` выбирает это значение из памяти, попутно скорректировав его смещением 4, и кладёт в тот же регистр `t1`.

По метке `addr` мы положили метку `var`, то есть адрес `0x10000004`.

Этот адрес оказывается в регистре `t2` тем же способом, каким `0xdeadbeef` оказалось в `t1`.

После чего с помощью явно указанного смещения в инструкции (не псевдо) `lw` получаем в разных регистрах содержимое памяти по адресам `0x10000004`, `0x10000008` и `0x10000000` соответственно.

При записи регистра в память трюк с временным хранением адреса в том же самом регистре не срабатывает: там лежит записываемое значение. Поэтому приходится использовать вспомогательный регистр (в примере — `t0`).

В программистских задачах часто приходится работать с записанными в память последовательностями однотипных данных — массивами.

Массив — это набор элементов данных одного размера, расположенных в оперативной памяти строго подряд.

Для работы с массивом необходима знать адрес начала массива в памяти, и длину массива — количество элементов * размер одного элемента. Обращение к элементу массива № `k` — это обращение к памяти по адресу `адрес + размер * k`. Например, 42-й элемент целочисленного массива по адресу `0x10000020` это четыре байта (одно машинное слово) по адресу `0x10000020 + 4 * 42 = 0x100000c8`.

Если нужно обработать массив данных, базовая или косвенная адресация — единственный способ. В примере 4 массив слов заполняется последовательными значениями:

```
.data
array: .word 64          # 64 слова
arrend:                          # Граница массива
.text
        la      t0, array  # Счётчик
        la      t1, arrend
        li      t2, 1      # Число, которое мы будем
                           # записывать в массив
loop:   sw      t2, 0(t0)   # Запись числа по адресу в t0
        addi    t2, t2, 1   # Изменим число
        addi    t0, t0, 4   # Увеличим адрес на размер
                           # слова в байтах
        bltu    t0, t1, loop # Если не вышли за границу массива
        la      t0, array
out:    lw      a0, 0(t0)   # очередной элемент массива
        addi    t0, t0, 4
        blt     t0, t1, out
```

Если мы хотим хранить в массиве `array` байты, а не слова, то между строкой из байтов и `array`: нужно добавить выравнивание на границу слова (`.align 4`). Так как сейчас там `.word`, то выравнивание произошло автоматически.

Задание.

Пусть у нас есть массив из 127 слов, содержащий беззнаковые целые числа. Напишите на языке ассемблера RISC-V программу для подсчёта суммы элементов этого массива. В случае, если сумма окажется больше, меньше или равна 16384, программа должна выдать на экран соответствующий ответ. Постарайтесь уменьшить число задействованных регистров процессора.

Контрольные вопросы:

1. На сколько четырёхбайтных инструкций можно перейти командой `j` (число может быть отрицательным, младший бит равен нулю и не хранится)?
2. Какой тип цикла был приведён в примере 3 этой практической работы?